Lesson 16 Handout – SARSA(λ)
Reinforcement Learning DSOR 646

This handout shows a Python implementation of the SARSA(λ) algorithm, an on-policy temporal difference (TD) control solution procedure. The pseudocode for the algorithm is displayed on page 305 of the course textbook. A linear value function approximation scheme, employing a tile coding approach, is applied to represent the value of state-action variables. The algorithm determines a superlative policy and value function for the *MountainCar-vo* Farama Foundation Gymnasium Environment. The provided code generates a graph displaying the *learning curve* and other important performance statistics associated with a particular hyperparameter design run, including the algorithm score and superlative policy EETDR.

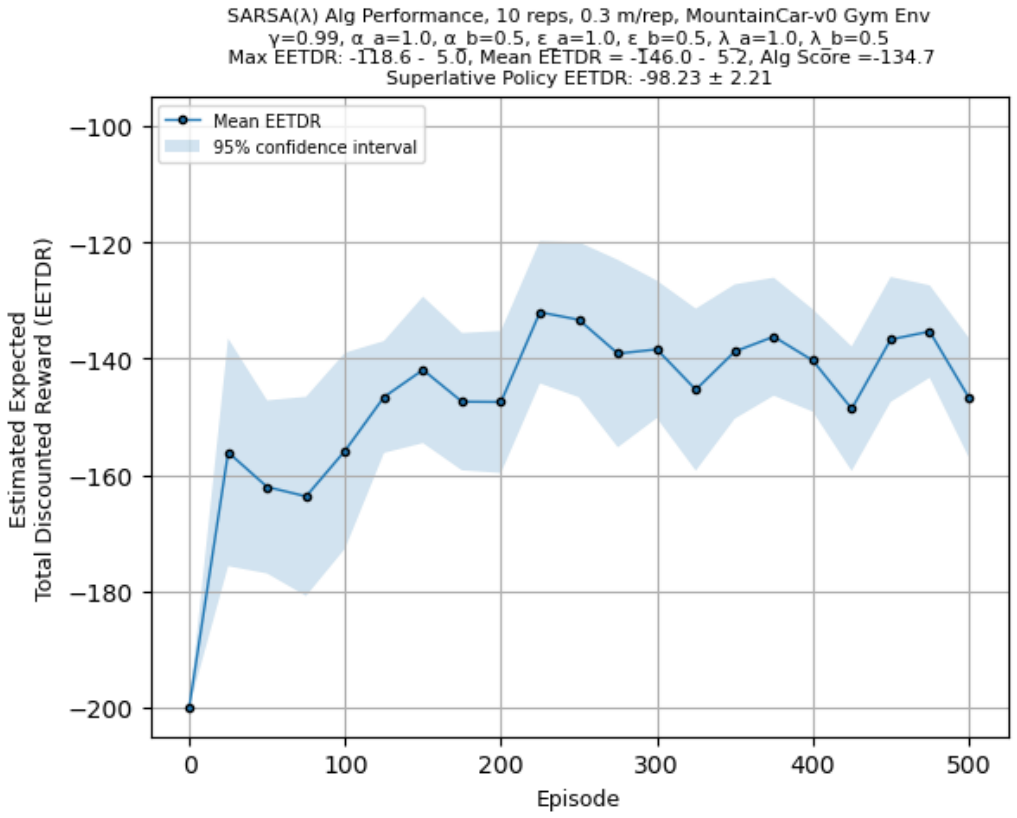

Figure 1: SARSA(λ) learning curve for the Farama Foundation Gymnasium MountainCar-vo Environment.

```
"""
DSOR 646 - Reinforcement Learning
Dr. Matthew Robbins
Lesson 16 Handout -- SARSA(lambda) with linear tile coding VFA scheme
Farama Foundation Gymnasium MountainCar-v0 Environment
"""

import gymnasium as gym
import numpy as np
```

```
import time
from scipy.stats import t
import matplotlib.pyplot as plt
from tiles3 import tiles, IHT
from sklearn import metrics
import copy

# warning suppression if using Python 3.12
import warnings
warnings.filterwarnings("ignore",category=UserWarning)

# create Farama Foundation Gymnasium environment
envName = "MountainCar-v0"
env = gym.make(envName)

print(f"\nFarama Foundation Gymnasium version: {gym.__version__}")
print(f"Environment: {envName}")

# Observation and action space
print("Observation space: {}".format(env.observation_space))
print("Action space: {}".format(env.action_space))

""" Algorithm Hyperparameters """
# algorithm name
algName = "SARSA(lambda)"

# VFA tile coding scheme for state-action variables
# max size of integer hash table (iht)
  # too small will impact performance
  # too large will impact computational time
max_size_iht = 2**10               # 10
# size parameters for tilings and tiles per tiling
num_tilings = 4                    # 4 partitions
num_tiles_per_tiling = 4           # 4 elements per partition

# VFA coefficient smoothing rule
alpha_a = 1.0                      # (*)   # 1.0
alpha_b = 0.5                      # (*)   # 0.5
def alpha(n):
    return (alpha_a/(1+n)**alpha_b)/num_tilings

# eps-greedy stepsize rule
eps_a = 1.0                        # (*)   # 1.0
eps_b = 0.5                        # (*)   # 0.5
def epsilon(n):
    return eps_a/(1+n)**eps_b

# trace decay parameter
lambda_df_a = 1.0                  # (*)   # 1.0
lambda_df_b = 0.5                  # (*)   # 0.5
def lambda_df(n):
    return lambda_df_a/(1+n)**lambda_df_b

# TD error clip (for stability)
  # (1/20 to 1/5 of range of q-values)
ʌ_clip = 10                    # 10

# discount factor
gamma = 0.99

# number of algorithm runs
Z = 10

# number of episodes
```

```
74  M = int(0.5e3)
75
76  # policy evaluation test frequency
77  test_freq = 25
78
79  # number of replications per test
80  num_test_reps = 30
81
82  # random number generator seed manual offset
83  offset = 0
84
85  # dimensionality of state space
86  num_state_dims = len(env.observation_space.low)
87
88  # discrete action space -- number of actions
89  num_actions = env.action_space.n
90
91  # define reference points in state space for use with VFA
92  Slow = np.array(env.observation_space.low)
93  Shigh = np.array(env.observation_space.high)
94  # overwrite environment infinite boundaries with finite ones
95  # Slow = np.array([])
96  # Shigh = np.array([])
97  Srange = Shigh-Slow
98  scaleFactor = num_tiles_per_tiling/Srange
99
100 # basis function reports active tiles for (s,a) pair
101 def phi(s,a,iht):
102     return tiles(iht,num_tilings,list(s*scaleFactor),[int(a)])
103
104 # approximate Q-function
105 def Qbar(s,a,w,iht):
106     qhat = 0
107     tiles = phi(s,a,iht)
108     for tile in tiles:
109         qhat += w[tile]
110     return qhat
111
112 # gradient of approximate Q-function
113 def gradQbar(s,a,iht): # returns index numbers of active tiles
114     return phi(s,a,iht)
115
116 # computes action that maximizes performance, as represented by approximate
117 #   Q-function, comprised of weight vector w and integer hash table
118 def argmaxQbar(s,w,iht):
119     Qvals = [Qbar(s,a,w,iht) for a in range(num_actions)]
120     return np.argmax(Qvals)
121
122 # function for computing confidence interval
123 def confinterval(data,alpha=0.05):
124     n = np.size(data)            # number of data points
125     se = np.std(data,ddof=1)/np.sqrt(n) # standard error
126     ts = t.ppf(1-alpha/2, n-1)   # t-score for two-tail test
127     mean = np.mean(data)
128     halfwidth = ts*se
129     return mean, halfwidth
130
131 # define policy evaluation function
132 def evaluate_policy(w,iht, num_reps, seed_mult=1):
133     # initialize test data structure
134     test_data = np.zeros((num_reps))
135     # run num_test_reps replications per test
136     for rep in range(num_reps):
137         # initialize episode complete flags
```

```
        terminated = False
        truncated = False
        # initialize episode reward
        Gtest = 0
        # initialize the system by resetting the enivronment, obtain state var
        state = env.reset(seed=seed_mult*1000+rep)[0]
        while not(terminated or truncated):
            # take action according to current policy, using theta
            action = argmaxQbar(state,w,iht)
            # apply action and observe system information
            state, reward, terminated, truncated, _ = env.step(action)
            # update episode cumulative reward
            Gtest += reward
        test_data[rep] = Gtest
    mean, hw = confinterval(test_data)
    return mean, hw

# initialize the data structure to hold the top 2Z scores and VFA parameters
num_best_scores = 2*Z
best_scores = [{'ETDR': -np.inf,'ETDR_hw': np.inf, 'w': None, 'iht': None} for _ in ...
    range(num_best_scores)]

# retain Top 2Z milestone validation testing results for final independent
#   testing to determine superlative policy across all replications
def update_best_scores(mean, hw, w, iht, best_scores):
    # Find the first score that mean is greater than
    for i in range(len(best_scores)):
        if mean > best_scores[i]['ETDR']:
            # Shift scores and parameters
            best_scores.insert(i, {'ETDR': np.copy(mean), 'ETDR_hw': np.copy(hw), 'w': ...
                np.copy(w), 'iht': copy.deepcopy(iht)})
            # We only want the top scores, so remove the last one
            best_scores.pop()
            return True
    return False

# record each episode's cumulative reward to measure online performance
Gzm = []
GzmTest = []

# start timer for executing algorithm design run
tic = time.perf_counter()

# loop through each algorithm run
for z in range(Z):
    # initialize theta vector of basis function weights
    w = np.zeros((max_size_iht,1))
    # initialize eligibility trace vector
    zeta = np.zeros((max_size_iht,1))
    # initialize state-action counter
    Nsa = np.zeros((max_size_iht,1))
    # initialize integer hash table
    iht_VFA = IHT(max_size_iht)

    print(f"\nSARSA(alpha_a={alpha_a:<3.2f},alpha_b={alpha_b:<3.2f},eps_a={eps_a:<3.2f},eps_b={eps_b:<3.2f},lambd
        rep {z}...")
    print(" Policy evaluation milestone validation tests...")

    # loop through each episode
    for m in range(M):
        # initialize episode complete flag (when system enters terminal state)
        terminated = False
        truncated = False
        # initialize episode reward
```

```
        Gm = 0
        # set random number generator seed
        np.random.seed(int(z*1e6+m+1e5*offset))
        # initialize the system by resetting the environment, obtain state var
        state = env.reset(seed=int(z*1e6+m))[0]
        # select action based on epsilon-greedy exploration mechanism
        if np.random.rand() > epsilon(m):
            # act greedy by exploiting currrent knowledge
            # take best action at current state using Qbar
            action = argmaxQbar(state,w,iht_VFA)
        else:
            # act randomly with probability epsilon to explore
            action = np.random.randint(0,num_actions)

        # SARSA main loop
        while not(terminated or truncated):
            # apply action and observe system information
            next_state, reward, terminated, truncated, _  = env.step(action)

            # update episode cumulative reward
            Gm += reward

            # select action based on epsilon-greedy exploration mechanism
            if np.random.rand() > epsilon(m):
                # act greedy by exploiting currrent knowledge
                # take best action at current state using Qbar
                next_action = argmaxQbar(next_state,w,iht_VFA)
            else:
                # act randomly with probability epsilon to explore
                next_action = np.random.randint(0,num_actions)

            # Compute qhat (i.e., target)
            qhat = reward + (1-terminated)*gamma*Qbar(next_state,next_action,w,iht_VFA)

            # compute TD error (clip for stability)
            δ = np.clip(-δ_clip, qhat - Qbar(state,action,w,iht_VFA),δ_clip)

            # Update elgibility trace
            zeta = gamma*lambda_df(m)*zeta
            active_tiles = gradQbar(state,action,iht_VFA)
            zeta[active_tiles] += 1

            # update state-action counter
            Nsa[active_tiles] += 1

            # update w vector
            w += alpha(Nsa)*δ*zeta

            # update state and action
            state = np.copy(next_state)
            action = np.copy(next_action)

            # update state variable boundary information
            # Slow_obs[state < Slow_obs] = state[state < Slow_obs]
            # Shigh_obs[state > Shigh_obs] = state[state > Shigh_obs]

        # record performance
        # print(f"\n   Episode: {m}, Cumulative reward: {Gm}")
        Gzm.append((z,Gm))

        # test current policy (as represented by current w and iht) every test_freq episodes
        if m % test_freq == 0:
            mean, hw = evaluate_policy(w, iht_VFA, num_test_reps)
            GzmTest.append((z, m, mean, hw))
```

```
263
264             # update best scores if necessary
265             if update_best_scores(mean, hw, w, iht_VFA, best_scores):
266                 print(f"   Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}  *** ...
                        New Top {num_best_scores} Policy -- w, iht recorded")
267             else:
268                 print(f"   Test... Episode: {m:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}")
269
270     # last test of current algorithm run
271     mean, hw = evaluate_policy(w,iht_VFA, num_test_reps)
272     GzmTest.append((z,M,mean,hw))
273
274     # update best scores if necessary
275     if update_best_scores(mean, hw, w, iht_VFA, best_scores):
276         print(f"   Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}  *** New Top ...
                {num_best_scores} Policy -- w, iht recorded")
277     else:
278         print(f"   Test... Episode: {M:>4}, EETDR CI: {mean:>6.1f} +/- {hw:4.1f}\n    ...
                *--------* Current Top 5 EETDRs: {best_scores[0]['ETDR']:>6.1f}, ...
                {best_scores[1]['ETDR']:>6.1f}, {best_scores[2]['ETDR']:>6.1f}, ...
                {best_scores[3]['ETDR']:>6.1f}, {best_scores[4]['ETDR']:>6.1f}")
279
280
281 toc = time.perf_counter()
282 print(f"\nExecuted {Z} algorithm reps in {toc - tic:0.4f} seconds.")
283
284 print(f"\nIdentify Superlative Policy via final independent testing of Top {num_best_scores} ...
        policies from milestone validation testing...")
285
286 # initialize list of means and half-widths for testing top policies
287 mean_values = []
288 hw_values = []
289
290 # loop through top policies stored in best_scores to find superlative policy
291 # via final testing with different random number seed and more reps
292 for i, score in enumerate(best_scores):
293     mean, hw = evaluate_policy(score['w'], score['iht'], 2*num_test_reps,2)
294     print(f"   Rank: {i+1:>2d} policy final test... EETDR CI: {mean:>6.2f} +/- {hw:>4.2f}")
295     mean_values.append(mean)
296     hw_values.append(hw)
297
298 # determine superlative policy and record its mean and half-width
299 indSupETDR = np.argmax(np.array(mean_values))
300 supETDR = mean_values[indSupETDR]
301 supETDRhw = hw_values[indSupETDR]
302
303 # VFA parameter-values representing superlative policy for algorithm design run
304 w = np.copy(best_scores[indSupETDR]['w'])
305 iht_VFA = copy.deepcopy(best_scores[indSupETDR]['iht'])
306
307 """
308 plot performance results
309 """
310 # organize validation milestone testing data for displaying learning curve
311 npGzmTest = np.array(GzmTest)
312 TestEETDR = np.reshape(npGzmTest[:,2],(Z,int(npGzmTest.shape[0]/Z)))
313
314 # policy evaluation via milestone testing (validation) -- Learning Curve
315 avgTestEETDR = np.mean(TestEETDR,axis=0)          # mean at each milestone
316 avgTestSE = np.std(TestEETDR,axis=0)/np.sqrt(Z)
317 avgTestHW = t.ppf(1-0.05/2, Z-1)*avgTestSE        # half-width at each milestone
318
319 # measure peak algorithm performance across multiple replications
320 maxTestEETDR = np.max(TestEETDR,axis=1)
```

```
meanMaxTestEETDR = np.mean(maxTestEETDR)
maxTestSE = np.std(maxTestEETDR,ddof=1)/np.sqrt(Z)
maxTestME = t.ppf(0.95, Z-1)*maxTestSE

# measure average algorithm performance across multiple replications
AULC = [metrics.auc(np.arange(0,M+1,test_freq), TestEETDR[z,:])/M for z in range(Z)]
meanAULC = np.round(np.mean(AULC),1)
meAULC = t.ppf(0.95, Z-1)*np.std(AULC,ddof=1)/np.sqrt(Z)

# algorithm score values hyperparameters resulting in both peak and and average performance
# peak performance valued slightly more than average performance
# reliablity valued by using lower bound of single-sided confidence interval, incorporating ...
    margin of error (ME)
AlgScore = 0.6*(meanMaxTestEETDR-maxTestME) + 0.4*(meanAULC-meAULC)

# create and show learning curve figure
plt.figure(0)
plt.plot(np.arange(0,M+1,test_freq),avgTestEETDR, marker = 'o', ms = 3, mec = 'k', linewidth ...
    = 1, label='Mean EETDR')
plt.fill_between(
    np.arange(0,M+1,test_freq),
    avgTestEETDR-avgTestHW,
    avgTestEETDR+avgTestHW,
    alpha=0.2,
    label='95% confidence interval')
plt.xlabel('Episode',fontsize=9)
plt.ylabel('Estimated Expected\nTotal Discounted Reward (EETDR)',fontsize=9)
plt.title(f'{algName} Alg Performance, {Z} reps, {np.round((toc-tic)/(Z)/60,1)} m/rep, ...
    {envName} Gym Env\ngamma={gamma}, alpha_a={alpha_a}, alpha_b={alpha_b}, eps_a={eps_a}, ...
    eps_b={eps_b}, lambda_a={lambda_df_a}, lambda_b={lambda_df_b}\n Max EETDR: ...
    {meanMaxTestEETDR:>6.1f} - {maxTestME:4.1f}, Mean EETDR = {meanAULC:>6.1f} - ...
    {meAULC:4.1f}, Alg Score ={AlgScore:6.1f}\nSuperlative Policy EETDR: {supETDR:>6.2f} +/- ...
    {supETDRhw:4.2f}',fontsize=8)
plt.legend(loc="upper left",fontsize=7)
plt.xlim([-0.05*M, 1.05*M])
plt.ylim([-205, -95])
plt.grid(which='both')
plt.show()
```